

퍼셉트론에서 트랜스포머까지: 다층 퍼셉트론·스케일 학습·LLM

Oct 1, 2026 · @김한서

책 2장의 퍼셉트론은 오늘날 LLM의 몸통이에요. 트랜스포머 파라미터의 약 3분의 2가 2층 퍼셉트론 (MLP) 구조이고, 이 층을 깊고 넓게 쌓아 데이터를 많이 넣는 것이 스케일 학습이에요.

1. 퍼셉트론과 단층의 한계 (책 2.1-2.4)

퍼셉트론은 입력에 가중치를 곱해 더하고, 그 합이 기준을 넘으면 1, 아니면 0을 내는 장치예요. 하나로써는 입력 공간을 **직선 하나**로만 나눌 수 있어서 XOR을 만들지 못해요.

$$y = 0 \quad (b + w_1 \cdot x_1 + w_2 \cdot x_2 \leq 0 \text{ 일 때})$$

$$y = 1 \quad (b + w_1 \cdot x_1 + w_2 \cdot x_2 > 0 \text{ 일 때})$$

게이트	(w1, w2, b) 예시	단층으로 가능?	이유
AND	(0.5, 0.5, -0.7)	가능	직선 하나로 (1,1)만 분리돼요
NAND	(-0.5, -0.5, 0.7)	가능	AND의 부호를 모두 뒤집은 것
OR	(0.5, 0.5, -0.2)	가능	직선 하나로 (0,0)만 분리돼요
XOR	없음	불가능	(0,1)·(1,0)과 (0,0)·(1,1)을 직선 하나로 나눌 수 없어요

- **핵심 통찰:** AND·NAND·OR는 구조가 같고 매개변수(w, b)만 다르죠. 같은 모델이 값만 바뀌 다른 일을 한다는 이 원리가 현대의 파인튜닝까지 이어져요.
- 직선으로 나뉘지는 영역을 **선형**, 곡선이 필요한 영역을 **비선형**이라고 해요(p.57).
- 이 한계는 1969년 Minsky와 Papert가 지적했고, 이후 신경망 연구가 한동안 위축되기도 했어요 ([Perceptrons \(book\)](#), [AI winter](#)).

2. 다층 퍼셉트론: 층을 쌓으면 표현력이 커져요 (책 2.5-2.6)

앞층 퍼셉트론들의 **출력값**이 다음 층의 **입력 변수**가 되도록 쌓으면, 단층으로 못 하던 XOR도 만들 수 있어요.

```
def XOR(x1, x2):
    s1 = NAND(x1, x2)    # 1층
    s2 = OR(x1, x2)       # 1층
    y = AND(s1, s2)       # 2층: s1, s2가 새로운 입력 변수
    return y
```



책은 이를 **공장 조립라인**에 비유해요(p.60). 1층 작업자가 부품을 다듬어 넘기면, 2층 작업자가 그 부품으로 완성품을 만들어요. "층을 쌓아(깊게 하여) 더 다양한 것을 표현할 수 있다"가 딥(Deep)러닝의 "딥"이예요.

정확하게 이해하기 위한 세 가지

1. 넘겨주는 것은 **함수가 아니라 출력값**이에요. 층 전체로 보면 함수 안에 함수를 넣는 합성 함수 $y = f_2(f_1(x))$ 예요.
2. 한 층에 퍼셉트론이 여러 개 나란히 있어요. XOR의 1층에도 NAND와 OR가 동시에 있었고, 실제 신경망은 수백~수천 개를 행렬곱 $x @ W + b$ 로 한 번에 계산해요(3장).
3. 층 사이에 **비선형 함수가 있어야** 쌓는 의미가 있어요. 선형 함수 $h(x) = cx$ 를 세 번 겹치면 $y = c^3x$ 로, 층 하나와 같아져요(3장 3.2.6). 계단 함수·시그모이드·ReLU가 층 사이를 "꺾어 주기" 때문에 깊이가 의미를 가져요.

2층이면 임의의 함수를 표현할 수 있다 (p.61)

책은 "2층 퍼셉트론(정확히는 비선형인 시그모이드 함수를 활성화 함수로 쓴 것)이면 임의의 함수를 표현할 수 있다"고 해요. 이를 보편 근사 정리라고 해요([Universal approximation theorem](#)).

- "표현할 수 있다"와 "찾을 수 있다"는 달라요. 책도 "가중치를 적절히 설정하기란 너무 어렵다"고 해요.
- 그래서 실제로는 층을 깊게 쌓고, 가중치는 데이터로 자동으로 찾게 해요(4-5장).

3. 책의 퍼셉트론에서 현대 MLP로

현대의 다층 퍼셉트론(MLP)은 계단 함수를 매끄러운 함수로 바꾸고, 가중치를 자동으로 학습하는 다층 퍼셉트론이에요. 책의 3-5장이 이 변화를 하나씩 다뤘요.

항목	2장 퍼셉트론	현대 MLP	책에서 배우는 곳
출력을 정하는 방식	계단 함수 (0 또는 1)	시그모이드, ReLU, GELU 등 매끄러운 함수	3장
가중치 w, b	사람이 직접 정함	데이터로 자동 학습	4장
학습 방법	없음	오차역전파 (미분으로 수정 방향 계산)	5장

항목	2장 퍼셉트론	현대 MLP	책에서 배우는 곳
계산 방식	$\text{np.sum}(w * x) + b$	행렬곱 $x @ W + b$ (데이터·뉴런 여러 개를 한 번에)	3장
규모	뉴런 몇 개	한 층에 수천 개, 총 수십 개	—

4. 스케일 학습: 키우면 좋아지고, 그에는 규칙이 있어요

모델(파라미터), 데이터, 계산량을 함께 키우면 언어 모델의 오차가 **예측 가능한 곡선을 그리며** 줄어요. 책 p.61의 “층을 쌓으면 컴퓨터까지 만들 수 있다”는 직관이 현대에 와서 측정 가능한 법칙이 된 셈이에요.

무엇을 키우나

키우는 것	퍼셉트론 관점의 의미
파라미터 수	w와 b의 개수. 층을 깊게(층 수↑) 또는 넓게(층당 뉴런 수↑) 하면 늘어요
데이터 양	가중치를 고치는 데 쓰는 예제의 수. LLM에서는 토큰(글자 조각) 수로 세요
계산량	학습에 쓰는 총 연산량. 대략 “파라미터 × 데이터”에 비례해요

두 개의 핵심 연구

연구	결론	의미
Kaplan 외, OpenAI (2020)	모델 크기·데이터·계산량과 손실 사이에 규칙적인 관계(멱법칙)가 있다	큰 모델을 만들기 전에 작은 모델로 결과를 예측할 수 있게 됐어요
Chinchilla, DeepMind (2022)	같은 계산 예산이면 파라미터와 데이터를 함께 키워야 효율적. 대략 파라미터 1개당 토큰 20개	7B 모델이면 약 1,400억 토큰이 계산 효율상 균형점이에요

요즘 공개 모델들은 사용할 때의 비용(추론 비용)을 줄이려고 이보다 훨씬 많은 데이터로 작은 모델을 학습시키는 경우가 많아요. 또 크기만 키우기보다, 아래의 MoE처럼 **효율적으로 키우는 방법**과 답하기 전에 더 오래 생각하게 하는 방법(추론 모델)을 함께 쓰는 흐름이에요.

5. 트랜스포머 속 퍼셉트론: FFN(MLP) 블록

트랜스포머 블록 하나는 **어텐션**과 **FFN**(피드포워드 네트워크)으로 이루어져 있고, FFN이 바로 2장의 2층 퍼셉트론 구조예요([Attention Is All You Need](#)).

부분	하는 일	비유
어텐션	문장 속 다른 토큰 중 어디를 볼지 정해서 정보를 모아 요	회의에서 누구 말을 참고할지 정하 기
FFN (MLP)	모은 정보를 토큰마다 따로 가공해요	각자 자리로 돌아가 생각 정리하기

FFN은 정확히 2장의 2층 퍼셉트론 모양이에요

```
class FFN(nn.Module):
    def __init__(self, d):
        super().__init__()
        self.fc1 = nn.Linear(d, 4 * d)    # 1층: 넓게 펼치기 (예: 768 → 3072)
        self.act = nn.GELU()               # 계단 함수 대신 매끄러운 활성화 함수
        self.fc2 = nn.Linear(4 * d, d)    # 2층: 다시 모으기 (3072 → 768)

    def forward(self, x):
        return self.fc2(self.act(self.fc1(x)))
```

XOR 회로와 똑같이 **입력 → 중간층 → 출력** 모양이에요. 차이는 중간 뉴런이 2개가 아니라 수천 개이고, 가중치를 학습으로 찾는다는 점뿐이에요. 최신 LLM은 활성화 함수로 SwiGLU 변형을 많이 써요([GLU Variants](#)).

파라미터의 약 3분의 2가 퍼셉트론이에요

임베딩 차원을 d 라고 할 때 블록 하나의 파라미터를 세면(편향 제외) 이렇게 돼요.

부분	계산	파라미터
어텐션	Q, K, V , 출력 행렬 4개 $\times (d \times d)$	$4d^2$
FFN	$(d \times 4d) + (4d \times d)$	$8d^2$ (약 2/3)

검산: GPT-2 small ($d = 768$, 블록 12개): 블록당 $12 \times 768^2 \approx 708$ 만 개, 12블록이면 약 8,500만 개예요. 단어 임베딩 약 3,900만 개를 더하면 총 **약 1억 2,400만 개(124M)**로, 공개된 GPT-2 small 크기와 맞아요.

FFN은 "지식 저장소" 역할을 해요

FFN 층은 일종의 **키-값 메모리**처럼 작동해요. 1층(fc1)의 각 뉴런이 특정 패턴을 감지하는 키, 2층(fc2)이 그에 맞는 다음 단어 분포 같은 값을 내보내요([Geva 외, EMNLP 2021](#)). "모델이 사실을 기억한다"고 할 때 그 지식의 상당 부분이 이 퍼셉트론 층에 있어요.

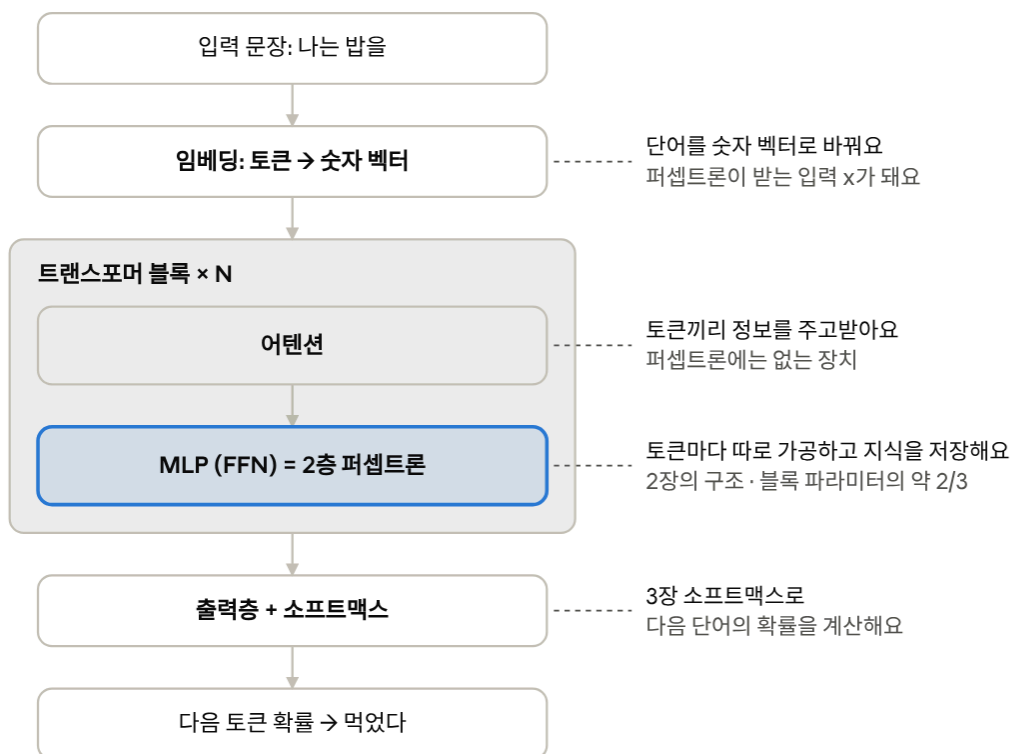
최신 트렌드: MoE (전문가 혼합)

FFN을 여러 개(전문가) 두고 **토큰마다 일부만 골라 쓰는** 방식이에요. Mixtral 8x7B는 블록마다 FFN 8개 중 2개만 써서, 전체 약 470억 개 파라미터 중 토큰당 약 130억 개만 사용해요([Mixtral of Experts](#)). "파라미터는 크게, 계산은 적게"라는 효율적 스케일링이고, 여기서 늘리는 대상이 바로 퍼셉트론 층이에요.

6. 언어 모델은 어떻게 구성되나

언어 모델은 "**어텐션 + MLP**"로 된 트랜스포머 블록을 수십 층 쌓고, 마지막에 다음 단어의 확률을 출력하는 모델이에요. GPT-2 small은 블록 12개, 요즘 대형 모델은 수십~100개 이상을 쌓아요.

언어 모델은 어텐션과 MLP 블록을 N번 쌓은 구조예요



언어 모델 구조 · 임베딩 → 블록 × N → 소프트맥스

블록 안에서는 어텐션(토큰끼리 소통)과 MLP(각자 가공)가 번갈아 반복돼요. 이 흐름은 [The Illustrated GPT-2](#)에 그림으로 자세히 나와 있어요.

MLP만으로는 왜 부족할까

MLP는 **단어 하나하나를 따로** 처리해요. "밥을" 위치의 MLP는 앞의 "나는"이 무엇이었는지 볼 수 없어요. 문장은 단어 사이의 관계가 핵심이라서, 단어끼리 정보를 주고받게 하는 어텐션이 필요해요.

구성 요소	퍼셉트론과의 관계	책에서
임베딩	단어를 숫자 벡터로 바꾸는 표. 퍼셉트론의 입력 x 를 만들어요	시리즈 2권
어텐션	내부에 가중합($x @ w$)은 쓰지만, 퍼셉트론에는 없는 장치	시리즈 2권
MLP (FFN)	2장의 2층 퍼셉트론 구조 그대로	2-3장
출력층 + 소프트맥스	마지막 퍼셉트론 층 + 확률로 바꾸는 함수	3장

정리하면 파라미터의 약 3분의 2가 MLP라서 **퍼셉트론이 몸통**인 건 맞지만, 언어를 이해하게 만드는 결정적 장치는 **어텐션**이에요. 이 책의 2-5장을 끝내면 MLP 부분은 모두 이해할 수 있고, 어텐션은 시리즈 2권(자연어 처리 편)에서 다루요.

7. AI 활용·에이전트와의 연결

AI를 쓰기만 한다면 수식까지 알 필요는 없어요. 하지만 구조를 알면 **AI의 행동을 예측**할 수 있어서, 결과를 믿어도 될지 판단하고 원하는 대로 통제하는 데 차이가 나요. AI가 대신해 주는 것은 "작성"이지 "판단"이 아니에요.

배운 개념	그래서 AI는 이렇게 행동해요	활용법
출력층은 다음 단어 확률을 계산	그럴듯한 단어를 고르는 것이라 모르는 것도 자신 있게 지어낼 수 있어요(환각) (OpenAI)	중요한 사실은 출처를 요청하고, "모르면 모른다고 해도 된다"고 허락해요 (Claude Docs)
학습이 끝나면 w, b 는 고정	대화한다고 모델이 새로 배우지 않고, 학습 이후 정보는 모르는 게 정상이에요	최신 문서·내 자료를 입력(x)에 직접 넣어요
지식은 MLP 가중치에 압축 저장	드문 지식은 흐릿하게 섞여 나와요. 바이트 코딩에서 오래된 함수가 나오는 이유예요	쓰는 버전과 공식 문서를 같이 줘요
입력은 토큰(숫자 벡터)으로 바뀜	글자 수 세기·철자 다루기에 약해요	"코드로 계산해줘"라고 요청해요
어텐션은 입력 안에서만 정보를 찾음	대화 밖 정보는 못 보고, 입력이 길면 핵심을 놓칠 수 있어요	필요한 자료만 골라 넣어요

배운 개념	그래서 AI는 이렇게 행동해요	활용법
파라미터 수 $\times$ dtype = 메모리	모델 크기가 속도·비용·로컬 실행 가능 여부를 정해요	7B $\times$ 2바이트 $\approx$ 14GB처럼 직접 계산해요
값(w) 바꾸기 vs 입력(x) 바꾸기	파인튜닝은 비싸고, 프롬프트·자료 넣기는 싸요	대부분 x를 바꿔 해결하고, 파인튜닝은 마지막 수단으로 뒹요

내 AI 에이전트를 만들고 업데이트할 때

에이전트는 새 모델이 아니라, 학습된 LLM에 도구·메모리·반복 루프를 붙인 **프로그램**이에요([Building effective agents](#)). 그래서 "에이전트를 업데이트한다"는 말은 대부분 가중치가 아니라 바깥 장치를 고치는 일이에요.

구성 요소	업데이트 방법	필요한 지식
LLM	보통 더 나은 모델로 교체	파라미터·비용 감각
지시문(시스템 프롬프트)	글을 고쳐요	"가중치는 고정, 입력으로 행동을 바꾼다"
도구	파이썬 함수를 추가·수정	1장 함수·딕셔너리(JSON) (도구 사용 문서)
메모리·자료	저장소나 검색 방식을 바꿔요	어텐션 감각: 필요한 것만 넣기 (Context engineering)
반복 루프·검증	코드를 고쳐요	확률적 출력 $\rightarrow$ 확인 단계 필수

하려는 일	필요한 수준
AI를 일상·업무에 활용	개념만: 다음 단어 예측, 가중치 고정, 입력으로 정보 주기
내 에이전트·AI 기능 만들기	1장 파이썬 + 2-3장 개념 + 코드 읽기
모델 자체를 학습·파인튜닝	책 전체 (4-5장 미분·오차역전파까지)

8. 스터디 토론 질문과 참고 자료

토론 질문

- 책은 "2층이면 컴퓨터도 만들 수 있다"고 했는데, LLM은 왜 블록을 수십 개나 쌓을까? (힌트: "표현할 수 있다"와 "학습으로 찾기 쉽다"의 차이)
- 층 사이에 비선형 함수가 없으면 100층을 쌓아도 왜 1층과 같아질까?

3. AND와 OR가 같은 구조, 다른 값이라는 사실은 파인튜닝과 프롬프트 엔지니어링의 차이를 어떻게 설명해 줄까?
4. MoE는 파라미터를 늘리면서도 계산을 줄이는데, 대신 잃는 것은 무엇일까? (힌트: 메모리)
5. 내가 만들고 싶은 AI 에이전트가 있다면, 모델·지시문·도구·메모리 중 무엇을 먼저 설계해야 할까?

실습 추천

- ☐ 2장의 XOR을 NumPy로 구현하고, 같은 것을 PyTorch `nn.Linear` 두 개로 바꿔 대응 관계를 주석으로 달기 (AI에게 요청해도 좋아요)
- ☐ GPT-2 small 파라미터 수(약 124M)를 $d = 768$, 블록 12개로 직접 계산해 보기

참고 자료

- 「밑바닥부터 시작하는 딥러닝」 2장 퍼셉트론 (p.47-62), 3장 3.2.6 비선형 함수
- [Perceptrons \(book\)](#) · [AI winter](#) · [Universal approximation theorem](#)
- [Scaling Laws for Neural Language Models \(Kaplan 외, 2020\)](#)
- [Training Compute-Optimal Large Language Models \(Chinchilla, 2022\)](#)
- [Attention Is All You Need \(2017\)](#) · [The Illustrated GPT-2](#)
- [GLU Variants Improve Transformer](#)
- [Transformer Feed-Forward Layers Are Key-Value Memories](#)
- [Mixtral of Experts](#)
- [Why language models hallucinate \(OpenAI\)](#) · [Reduce hallucinations \(Claude Docs\)](#)
- [Building effective agents](#) · [Effective context engineering for AI agents](#) · [Tool use 문서](#)